Before an AI System Answers Anything, a Language Model Knows How Well It Will Do

Siwei Shan[*]
NLPQuant AI Inc.

Sunny Yang[*]
University of Illinois Urbana-Champaign
guangy4@illinois.edu

Weiyan Zhao
NLPQuant AI Inc.

Abstract

Given an AI system's graded answers to a few questions of a benchmark, we predict whether it answers each remaining question correctly. The hardest case is before it has answered any, because the predictor does not yet know how strong the system is. A language model such as Claude does. We give it the system's name and a few of the benchmark's questions, ask for the system's accuracy, and use that as the starting probability that the system's answers then update. On 44 public benchmarks that no part of our development touched, the estimate's typical error is 13 accuracy points, against 25 for guessing 50%, and it is as accurate as eight of the system's own answers. On the hidden benchmarks of the Predictive AI Evaluation Competition (PAIEC), starting from it lowered the error before the first answer on six of the seven benchmarks. It is the largest single gain in our predictor. Without the system's name it is barely better than a guess.

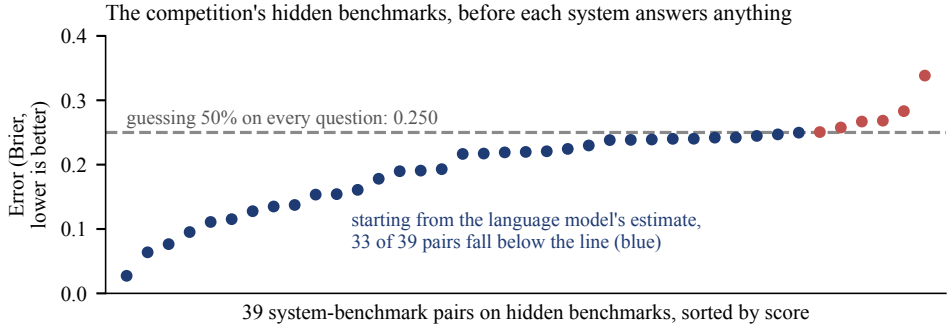


Figure 1: Error before the first answer on the competition's hidden benchmarks. Each dot is one distinct system–benchmark pair from the five live runs that start from the language model's estimate. Three pairs appeared in more than one run and are averaged. Without the estimate the predictor answers one half and scores 0.250 on every pair. 33 of 39 pairs fall below that line (blue), 6 above it (red).

1 Introduction

The Predictive AI Evaluation Competition asks how likely an AI system is to answer each question (item) of an unseen benchmark correctly. We call a system under a given configuration a subject. Predictions are scored after the subject reveals 0, 1, 3, 7, 15 and 31 of its own answers, by the Brier area under that learning curve (ALC). Budgets 0 to 3 carry half of the weight, and that is where

[*]Equal contribution.

predictors lose most. With no answers yet, a predictor cannot tell a system that answers nearly everything from one that answers half.

Our idea is to ask a language model. It has read about most named AI systems, so it can say how often a given system will succeed before the system answers anything. We turn its estimate into a prior that the system's own answers then update. Our contributions:

- **The finding** (Section 4): a language model knows roughly how strong a named AI system is, and the knowledge comes from the system's identity.
- **The effect** (Sections 5 and 6): starting from its estimate helps most before the first answer, on the hidden benchmarks and on public ones untouched by development.
- **A reusable kit**: the predictor, a simulator of the competition protocol, every cached language-model output, and 1,157 accuracy forecasts (757 on the main public benchmarks, 400 on 44 newer ones) that other methods can be scored on.

Related work. The organisers' Bayesian Linguistic Evaluator also starts from a language model's estimate, but it asks for every target response separately and revises the estimate in the prompt. We ask once per subject and benchmark and update the estimate numerically. Language models have also been asked to forecast benchmark scores from task descriptions [9], and missing scores can be completed from a model's other scores [10]. We use such a forecast as the prior of an item-level predictor. Methods that predict from few items usually assume items already calibrated on many models [1–4], which a new benchmark lacks. In measurement terms, the task is a Rasch model [5, 6] fitted online from very few responses, as in adaptive testing [7], and our prior is the collateral information that sets an examinee's starting ability [8].

2 Task

For each subject–benchmark pair, items are split 50/50 into an acquisition pool and an evaluation pool. Acquisition items stream past, and the program decides which to reveal, up to 31. Predictions on the evaluation pool are scored at budgets $b \in \{0, 1, 3, 7, 15, 31\}$ with weights $0.1, 0.2, 0.2, 0.2, 0.2, 0.1$, and the ALC is averaged over pairs. The predictor sees the labels revealed for every pair in a run. The program sees item text and subject attributes, never reference answers.

3 Method

Subject prior. We ask Claude Opus 5.5 what share of the benchmark's items the subject answers correctly, given up to eight sample items and the subject's name, provider, release date, harness and reasoning effort. Eight solved examples from public benchmarks precede the question and fix the scale. The estimate q_s is shrunk toward one half, $m_s = 0.5 + 0.75\,(q_s - 0.5)$, and acts as a Beta prior with four pseudo-counts:

$$\hat{\mu}_s = \frac{\sum_{j \in R_s} y_{sj} + 4\, m_s}{|R_s| + 4}, \tag{1}$$

where R_s is the set of items s has revealed.

The rest of the predictor. It has five parts.

- *One subject on the benchmark.* With fewer than ten labels from other subjects, the predictor uses the subject's own labels: $\hat{\mu}_s$ above, or without a subject prior the success rate with two pseudo-counts at one half.
- *Item ratings.* Opus rates each item's difficulty. Before the first answer, without a subject prior, the rating is read as a probability and moved three quarters of the way back to one half. After that it adds a slope times the standardised rating, with the slope fitted by ridge regression on at most 60 revealed items. When other subjects share the benchmark, the slope is fitted on this benchmark's items alone.
- *Other subjects on the benchmark.* The subject's rate is pulled toward theirs with eight pseudo-counts. The subject prior shifts it by half the logit gap between its estimate and theirs, fading with the same pseudo-counts.

Table 1: Error of the estimated accuracy of a system on a benchmark, before the system answers anything. This is the RMSE of the accuracy estimate against the system's true accuracy, not the Brier score of Figures 1 and 2. Lower is better. The blocks are different samples, so compare rows only within a block. Full table in Appendix A.

Estimate	Error
277 pairs with an identifiable system, from 24 public benchmarks	
Guess 50%	0.197
Average accuracy of all systems on the other benchmarks	0.199
Language model (Opus 5.5, or Sonnet and Haiku where Opus gives no number)	**0.168**
400 pairs from 44 newer benchmarks that no part of our development touched	
Guess 50%	0.247
Language model (as above)	**0.133**
An earlier sample of 71 pairs	
Guess 50%	0.217
Opus 5.5	0.141
with the system's name, provider and date hidden	0.211
with the benchmark named in the prompt	0.136
Haiku 5.5	0.185

- *Similar and identical items.* Each item moves with the residuals of its 30 most similar revealed items, by character n-grams of the text. When other subjects answered the same item, one Newton step of a Rasch model adjusts its logit.
- *Acquisition.* The rule reveals items no other subject has revealed yet, and never looks at outcomes.

4 The language model knows the system

We measured the prior on public benchmarks, against each subject's true accuracy. We express its error as the number n of the subject's own answers whose Laplace estimate $(k+1)/(n+2)$ has the same mean squared error.

Table 1 shows the result. On the 400 pairs from 44 benchmarks that no part of our development touched, the estimate's typical error is 13 accuracy points against 25 for guessing one half, which makes it worth eight of the subject's own answers. On the 277 named pairs from the 24 public benchmarks that supply no calibration example, the deployed prior is worth four of the subject's own answers (90% interval 0 to 10, bootstrap over benchmarks). Priors from data alone do not work before the first answer: the average benchmark's accuracy is worth nothing on these pairs, and so is a subject's level fitted on its other public benchmarks.

Where the knowledge comes from. The bottom rows of Table 1 answer this. When we hide the subject's name, provider and release date, the estimate falls almost to the constant, so the value comes from knowing which system it is. When we name the benchmark, which the deployed prompt never does, the estimate barely changes, so we found no sign that it retrieves a stored score for this benchmark. A stronger language model knows more: Haiku is worth three answers, Sonnet and Opus seven. What it lacks is how hard the benchmark is, which is why a calibration example on the same benchmark raises its worth to twelve answers. Table 3 in Appendix A lists every answer count. A check on systems released after the language model's training cutoff is in Section 7.

5 The prior improves the predictor

To isolate the prior, we ran the submitted predictor on 90 simulated draws from the public benchmarks we developed on, the same draws for every variant, and switched its language-model parts off in turn. Each draw holds 3 to 8 subject–benchmark pairs (5.7 on average), mostly one subject per benchmark (Appendix B). The subject priors are real Opus calls with the deployed prompt, cached so every variant sees the same estimates.

Table 2: The submitted predictor on 90 simulated draws from the public benchmarks we developed on, the same draws for every variant. Differences are paired over draws ($\pm$ one standard error over draws), against the variant with both language-model parts off.

Variant	ALC	B_0	Difference	Better in
Laplace estimate on own answers	0.2020	0.250	$+0.0008 \pm 0.0002$	20% of draws
Neither part	0.2012	0.250	–	–
Item ratings only	0.1992	0.244	-0.0020 ± 0.0004	68% of draws
Subject prior only	0.1941	0.221	-0.0071 ± 0.0009	80% of draws
Both (submitted)	**0.1924**	0.216	-0.0088 ± 0.0010	83% of draws

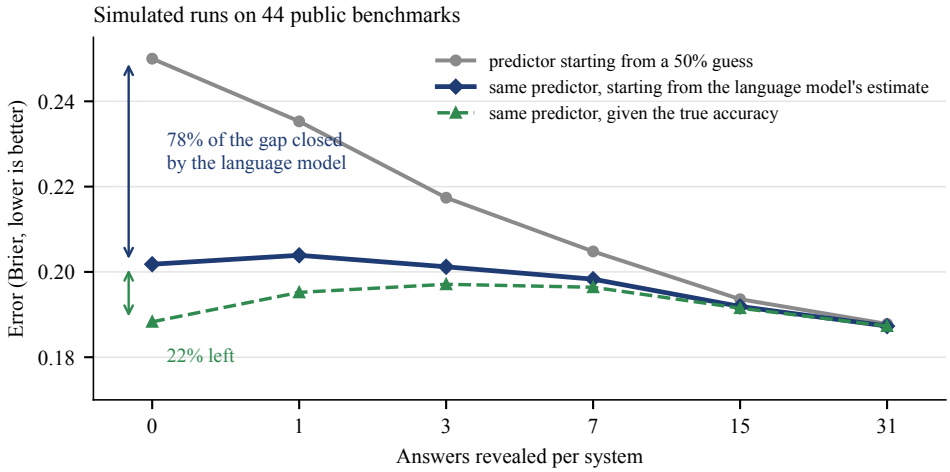


Figure 2: The submitted predictor on simulated draws from 44 public benchmarks that no part of our development touched (30 draws, 149 pairs with an identifiable system). It starts from a 50% guess, from the language model's estimate, or from each system's true accuracy, all through the same shrinkage toward one half. Before the first answer the language model closes 78% of the gap. With a good starting point the error rises over the first few answers, because the predictor lets them move it too far.

Table 2 gives the result. Figure 2 repeats the comparison on the 44 untouched benchmarks and adds the true accuracy as a starting point. With one subject per benchmark, the predictor without its language-model parts is barely better than a smoothed average of the subject's own answers. The prior is the largest part of the gain. It lowers the error at budget 0 from 0.250 to 0.221, and its effect fades by budget 7, once the subject's own answers take over.

What else we tried. We also tried to get more out of the item ratings and to learn the whole predictor from data. A fine-tuned difficulty model, a per-benchmark choice between rating sources, and one 4B network that reads every revealed answer gave no reliable improvement (Appendix D). The difficulty model's small gain came from a single safety benchmark, and it lost on three benchmarks it had never seen.

6 Live results

Each formative run draws nine subject–benchmark pairs from seven benchmarks, and in our runs most benchmarks had a single subject. Raw scores swing with the draw. Three runs of the same final package scored 0.130, 0.181 and 0.164. Budget 0 still allows a paired comparison. Without the prior, our predictor says one half for every item before any answers, so its Brier score at budget 0 is 0.250 on any draw. With the prior the score was 0.199 on the 39 distinct pairs of the five runs with the prior and without item ratings (90% interval 0.166 to 0.227, bootstrap over the seven hidden benchmarks), lower on 33 of them. Three pairs appeared in more than one run, and we average them. On these pairs the prior helped on average on six of the seven hidden benchmarks. On the seventh it

made the error worse ($+0.018$, on three of its five pairs). The three runs of the final package, which add Opus item ratings, scored 0.187 on their 26 pairs. The feedback identifies benchmarks only by an anonymous code, so we cannot say which benchmark that is or why. Later budgets offer no such paired comparison. Appendix C lists every run.

7 Limitations

The prior depends on the language model knowing the subject. For unnamed subjects the predictor leaves it out. In simulation on a further 15 benchmarks never used in development the prior made no measurable difference: -0.0011 on those with named subjects (90% interval -0.009 to $+0.007$), helping on the five where its estimate was closer to the true accuracy than one half is, and hurting on the three where it was further (Appendix B). On 44 newer benchmarks, also untouched by development and with identifiable systems, it lowered the ALC from 0.212 to 0.196 over 90 simulated draws (-0.016, 90% interval -0.019 to -0.013 by benchmark), better in every draw (Figure 2 shows the first 30). Systems there are further from one half, which makes a good starting estimate worth more (Appendix A). The benchmarks of Table 2 also served for development, so its gains may be optimistic. Online subjects were further from one half than those in the training data, and the shrinkage toward one half makes the prior too cautious for them. The live comparison spans different draws, and we did not compare against the organisers' Bayesian Linguistic Evaluator, whose default configuration failed on the current public data. We submitted a fix (paiec_baseline#4). The method needs a paid API key, and all benchmarks we used have binary responses. Most subjects were released before the language model's training cutoff. On three systems released after it, on one public agent benchmark, the prior beat guessing, but its advantage over the average of earlier systems came from one of the three.

8 Reproducibility and disclosures

Code, the simulator, cached language-model outputs and the logs behind every simulation are at `https://github.com/sunnyadn/paiec-pooled-priors`. `python reproduce.py` prints the simulation tables from the logs. With access to the gated measurement-db, the documented scripts rebuild the item files, after which `reproduce.py` recomputes Table 1 and `reproduce.py -rerun` replays the simulated draws from stored identifiers and reruns the simulations without an API key. The final Codabench submission is 1017682. The submission is two files that need only the Python standard library and NumPy. Because the platform recreates its worker processes at every budget, the predictor stores every language-model result on disk for all workers to share and caps calls per worker. Without an API key it runs on labels alone.

Disclosures. Training data: public measurement-db only, for development, simulation and the eight calibration examples. Pretrained models: Claude Opus 5.5 through the Anthropic API at test time, with Claude Sonnet 5.5 and Claude Haiku 5.5 as fallbacks. During development we also compared GPT and Gemini models. Development used LLM coding assistants (Claude Code and Codex).

References

[1] F. Maia Polo, L. Weber, L. Choshen, Y. Sun, G. Xu, M. Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. arXiv:2402.14992, 2024.

[2] S. Truong, Y. Tu, P. Liang, B. Li, S. Koyejo. Reliable and efficient amortized model-based evaluation. arXiv:2503.13335, 2025.

[3] V. Hofmann, D. Heineman, I. Magnusson, K. Lo, et al. Fluid language model benchmarking. arXiv:2509.11106, 2025.

[4] R. Vivek, K. Ethayarajh, D. Yang, D. Kiela. Anchor Points: benchmarking models with much fewer examples. arXiv:2309.08638, 2023.

[5] G. Rasch. Probabilistic Models for Some Intelligence and Attainment Tests. Danish Institute for Educational Research, 1960.

[6] F. M. Lord. Applications of Item Response Theory to Practical Testing Problems. Erlbaum, 1980.

[7] W. J. van der Linden and C. A. W. Glas, editors. Elements of Adaptive Testing. Springer, 2010.

[8] R. J. Mislevy. Exploiting auxiliary information about examinees in the estimation of item parameters. Applied Psychological Measurement, 11(1):81–91, 1987.

[9] J. Park, E. Mendes, G. Stanovsky, A. Ritter. Anticipatory evaluation of language models: predicting LLM benchmark scores from descriptions. arXiv:2509.20645, 2025.

[10] Y. Zeng, D. Papailiopoulos. You don't need to run every eval. arXiv:2606.24020, 2026.

A Prior quality in detail

Table 3: How well a language model estimates a subject's accuracy before any answers. The top rows use every named pair from the 24 public benchmarks that supply no calibration example, with the shrinkage fixed before this evaluation. The lower rows compare models on an earlier sample of 71 pairs. "Answers" is the number of the subject's own answers with the same mean squared error. *Uses other subjects' answers on the same benchmark, which do not exist before the first answer.

Estimate	Pairs	RMSE	Correlation	Answers
Constant 0.5	277	0.197	–	0
Mean accuracy of public benchmarks, this one left out	277	0.199	–	0
Subject's level from its other public benchmarks	277	0.203	–	0
Deployed: Opus, then Sonnet, then Haiku	277	**0.168**	0.55	**4**
Subject's level plus this benchmark's level from other subjects*	277	0.143	–	7
Constant 0.5	71	0.217	–	0
Claude Haiku 5.5	71	0.185	0.63	3
Claude Sonnet 5.5	71	0.142	0.76	7
Claude Opus 5.5	71	0.141	0.76	7
Opus, subject's identity hidden	71	0.211	0.28	1
Opus, benchmark named in the prompt	71	0.136	0.78	8

Benchmark sets. The simulations of Table 2 draw from all public benchmarks we developed on. Three further sets of public benchmarks evaluate the prior. The main evaluation has 32 benchmarks with named subjects, eight of which supply a calibration example, leaving the 24 of Table 1. Appendix B simulates 15 benchmarks used in no development step, eight of them with mostly named subjects. Figure 2 and the second block of Table 1 use 44 benchmarks added to the public data after development, with identifiable systems. Their subjects are further from one half (guessing scores 0.247 there against 0.197 on the main set), and the prior's error is about half that of guessing there, against 85% of it on the main set.

The full evaluation takes every public pair with a named subject and at least 50 responses, one subject per name and benchmark: 765 pairs on 32 benchmarks, of which the eight calibration pairs are left out. Every estimate saw five sample items (the deployed predictor first asks with the first item a worker sees, and asks once more when eight items of the benchmark have been seen), and the shrinkage of 0.75 was fixed before this evaluation. Opus gave a number on 678 pairs. On the other 79, all from `alignment_faking` and `harmbench`, the deployed predictor falls back to Sonnet and then Haiku. Table 4 splits the result. The data-only priors in Table 1 are fitted on the public response tables with the target pair left out. The subject's level and the benchmark's level are additive on the logit scale, fitted by alternating ridge regression; a subject seen on no other public benchmark (127 of the 277 pairs) gets level zero. The lower rows of Table 1 come from an earlier sample of at most four pairs per benchmark, using the 71 on which all three models gave a number.

B Benchmarks never used in development

The simulator of Section 5 samples seven benchmarks per draw with up to two subjects each and keeps a benchmark only if enough subjects answered 70% of its 120 sampled items. We repeated

Table 4: The deployed prior on every named public pair, with 90% intervals from a bootstrap over benchmarks.

Benchmarks	Pairs	RMSE	Correlation	Answers
All benchmarks	757	0.139 [0.121, 0.171]	0.67	8 [4, 11]
Benchmarks with a calibration example	480	0.119 [0.111, 0.129]	0.74	12 [9, 13]
Benchmarks without one	277	0.168 [0.128, 0.201]	0.55	4 [0, 10]
of which never used in development	32	0.177 [0.134, 0.215]	0.71	4 [1, 8]

the paired comparison on 15 benchmarks used in no development step, with item ratings off in both arms. Seven have mostly unnamed subjects, where the predictor leaves the prior out and the difference is zero by design. On the eight with mostly named subjects the prior changed the ALC by -0.0011 averaged over pairs (90% interval -0.009 to $+0.007$ from a bootstrap over benchmarks). Table 5 splits the change by budget. On every benchmark the change in ALC follows the change at budget 0: the prior helps where its starting estimate is closer than one half and hurts where it is further, and either way its effect is nearly gone by seven answers.

Table 5: The subject prior on the eight held-out benchmarks with named subjects: change in Brier score at budgets 0, 1, 3 and 7 and in ALC, prior minus no prior, over the same 90 draws (negative is better).

Benchmark	Pairs	B_0	B_1	B_3	B_7	ALC
`bertaqa`	27	-0.059	-0.041	-0.023	-0.009	-0.021
`evouna`	54	-0.050	-0.036	-0.012	-0.002	-0.015
`aider`	49	-0.029	-0.025	-0.018	-0.004	-0.013
`healthadminbench`	47	-0.035	-0.018	-0.006	-0.000	-0.008
`edu_circuit_hw`	46	-0.015	-0.010	-0.007	-0.003	-0.005
`clasheval`	51	$+0.017$	$+0.009$	$+0.006$	$+0.000$	$+0.005$
`dbpa`	59	$+0.051$	$+0.027$	$+0.013$	$+0.003$	$+0.014$
`ehrflowbench`	51	$+0.056$	$+0.035$	$+0.025$	$+0.012$	$+0.023$
All eight	384	-0.003				-0.001

C Online runs

Table 6: Every online formative run. Section 6 uses the five runs marked "+ subject prior". B_0 and B_{31} are the mean Brier scores at budgets 0 and 31, averaged over a run's pairs, and excess is the ALC above each pair's budget-31 level. The runs drew different subjects, so the rows are not a paired comparison.

Submission	Version	Pairs	B_0	Raw ALC	B_{31}	Excess
1016627	v6	9	0.250	0.158	0.116	0.042
1016628	v6	9	0.250	0.170	0.143	0.027
1016651	v6, no API key	8	0.250	0.158	0.134	0.024
1017498	+ subject prior (v19)	9	0.176	0.169	0.164	0.006
1017499	+ subject prior (v19)	9	0.177	0.159	0.151	0.008
1017545	+ subject prior (v23)	8	0.214	0.195	0.183	0.011
1017562	+ subject prior (v25)	8	0.195	0.179	0.169	0.010
1017629	+ subject prior (v27)	9	0.203	0.184	0.167	0.017
1017607	final	8	0.207	0.181	0.175	0.006
1017628	final	9	0.156	0.130	0.126	0.004
1017682	final (selected)	9	0.199	0.164	0.149	0.014

D Variants that did not improve the submission

Table 7: Variants tried and dropped. Differences in pair-averaged Brier ALC against the predictor of the same stage (negative is better). Item ratings help only where a benchmark's labels move with ability. On some safety benchmarks a label of 1 means that an attack succeeded, which reverses the rating's direction, and in our tests nothing in a benchmark's description told the two kinds apart.

Variant	Result
Opus and GPT prior averaged (simulation)	-0.0011 ± 0.0003
same, online (4 runs)	B_0 excess 0.048–0.085 vs 0.031–0.049
Text-learned difficulty (n-grams, embeddings, LoRA)	correlation ≤ 0.08
Prior ensembles (Opus twice, Opus with Sonnet)	RMSE 0.139–0.142 vs 0.141
Gemini added to the Opus and GPT prior	RMSE 0.137 vs 0.138
Later predictor, four held-out benchmarks of our development split	
Fine-tuned 4B difficulty model instead of Opus ratings	-0.0012 (one refusal benchmark)
same, three benchmarks it never saw	$+0.0032$
Two rating slopes, chosen per benchmark from the labels	-0.0003
Ratings off where a language model judges label 1 an attack	-0.0021 held out, $+0.0021$ training
One 4B network reading all revealed answers (126,556 sequences)	$+0.0085$ (500 draws)
No absolute rating before the first answer	-0.0007 held out, $+0.0006$ training
Ratings used only when the revealed labels agree with their direction	-0.0000